

**VIETNAM NATIONAL UNIVERSITY HO CHI
MINH CITY**

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS**

_____ oo0oo _____



Computer Architecture

Design of a Vending Machine

Supervisor:

Assoc. Prof. Dr. Tran Hoang Linh

Full Name	Student ID
Nguyễn Duy Tuyên	2213821
Nguyễn Tuệ	2213812
Phan Hoàng Minh Khôi	2211692

Ho Chi Minh City, April 21, 2026

Contents

1	Introduction	1
2	RTL Design Specification	1
2.1	FSM Archcitecture	2
2.2	State Machine Diagram	3
3	Verification Architecture	5
4	Test Scenarios & Simulation Results	6
4.1	Directed Test	6
4.2	Constrained Random Test	8
5	Conclusion	10

List of Figures

1	FSM Moore	2
2	FSM Control	4
3	FSM Control	5
4	Waveform Of Direct Test	8
5	Waveform Of Random Test	10

List of Tables

1	Signal Descriptions for the Vending Machine Module	1
2	Original FSM State Transition Table (13 States)	3
3	Optimized FSM State Transition Table (9 States)	3
4	Summary of Directed Test Scenarios	6
5	Directed Test Simulation Log Output (Partial)	7
6	Summary of Parameters and Results for the Random Test Phase	8
7	Constrained Random Test Simulation Log Output (Partial)	9

1 Introduction

This report outlines the design and verification of a Finite State Machine for an automated vending machine. The system features three input signals corresponding to coin insertions of 5, 10, and 25 cents. The system's outputs include a dispense signal and a 3-bit binary value representing the specific amount of change to be returned to the user.

2 RTL Design Specification

Before diving into the state machine logic, this section outlines the hardware interface of the Vending Machine module. The table below provides a comprehensive description of all input and output signals, including their types, bit-widths, and functional roles within the design.

Signal Name	Type	Width	Description
i_clk	Input	1	System clock for synchronous logic operations.
i_rst_n	Input	1	Asynchronous active-low reset signal.
i_nickle	Input	1	Asserted when a 5¢ coin (Nickel) is inserted.
i_dime	Input	1	Asserted when a 10¢ coin (Dime) is inserted.
i_quarter	Input	1	Asserted when a 25¢ coin (Quarter) is inserted[cite: 1, 5].
o_soda	Output	1	Asserted high when a product is successfully dispensed.
o_change[2:0]	Output	3	3-bit binary value representing the amount of change returned.

Table 1: Signal Descriptions for the Vending Machine Module

2.1 FSM Architecture

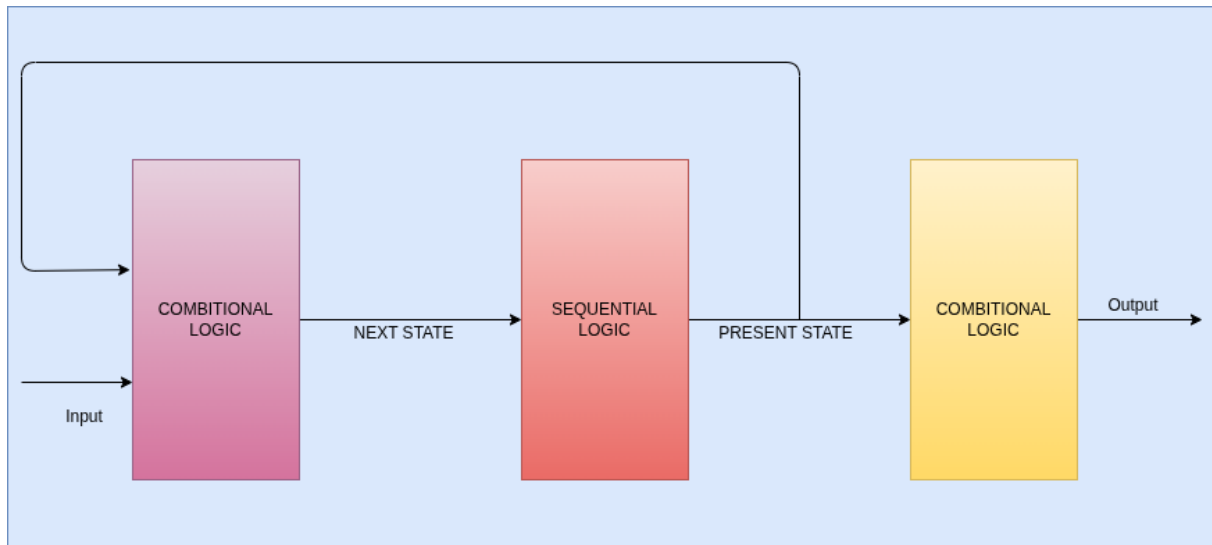


Figure 1: FSM Moore

This project employs a Moore Finite State Machine (FSM) architecture due to its simplicity, ease of implementation, and widespread adoption in hardware design. The architecture follows a standard state machine topology: external inputs feed into a combinational logic block to evaluate the 'Next State'. This value is then registered by a sequential logic block, which updates the 'Present State' at each clock cycle. A defining characteristic of the Moore FSM utilized in this design is that the system outputs depend exclusively on the Present State, whereas the Next State is a function of both the Present State and the external inputs.

2.2 State Machine Diagram

The following table details the transition logic of the initial 13-state FSM, covering every possible credit accumulation path before any state reduction was applied.

Present State	Input (N, D, Q)	Output (Soda, Change)	Next State
IDLE (0¢)	5¢/ 10¢/ 25¢	0 / 000	S5 / S10 / S25
S5 (5¢)	5¢/ 10¢/ 25¢	0 / 000	S10 / S15 / S30
S10 (10¢)	5¢/ 10¢/ 25¢	0 / 000	S15 / S20 / S35
S15 (15¢)	5¢/ 10¢/ 25¢	0 / 000	S20 / S25 / S40
S20 (20¢)	-	1 / 000	IDLE
S25 (25¢)	-	1 / 001	IDLE
S30 (30¢)	-	1 / 010	IDLE
S35 (35¢)	-	1 / 011	IDLE
S40 (40¢)	-	1 / 100	IDLE
... (Remaining intermediate transitions omitted for brevity) ...			

Table 2: Original FSM State Transition Table (13 States)

While the 13-state model provides a comprehensive logical map, it contains redundant transitions that increase hardware overhead. To optimize resource utilization and simplify the control logic, we applied state reduction techniques to merge equivalent states, resulting in a streamlined 9-state architecture that maintains full functional compliance with the original specifications.

Present State	Input (N, D, Q)	Output (Soda, Change)	Next State
IDLE	5¢/ 10¢/ 25¢	0 / 000	S5 / S10 / D25
S5	5¢/ 10¢/ 25¢	0 / 000	S10 / S15 / D30
S10	5¢/ 10¢/ 25¢	0 / 000	S15 / D20 / D35
S15	5¢/ 10¢/ 25¢	0 / 000	D20 / D25 / D40
D20	-	1 / 000	IDLE
D25	-	1 / 001	IDLE
D30	-	1 / 010	IDLE
D35	-	1 / 011	IDLE
D40	-	1 / 100	IDLE

Table 3: Optimized FSM State Transition Table (9 States)

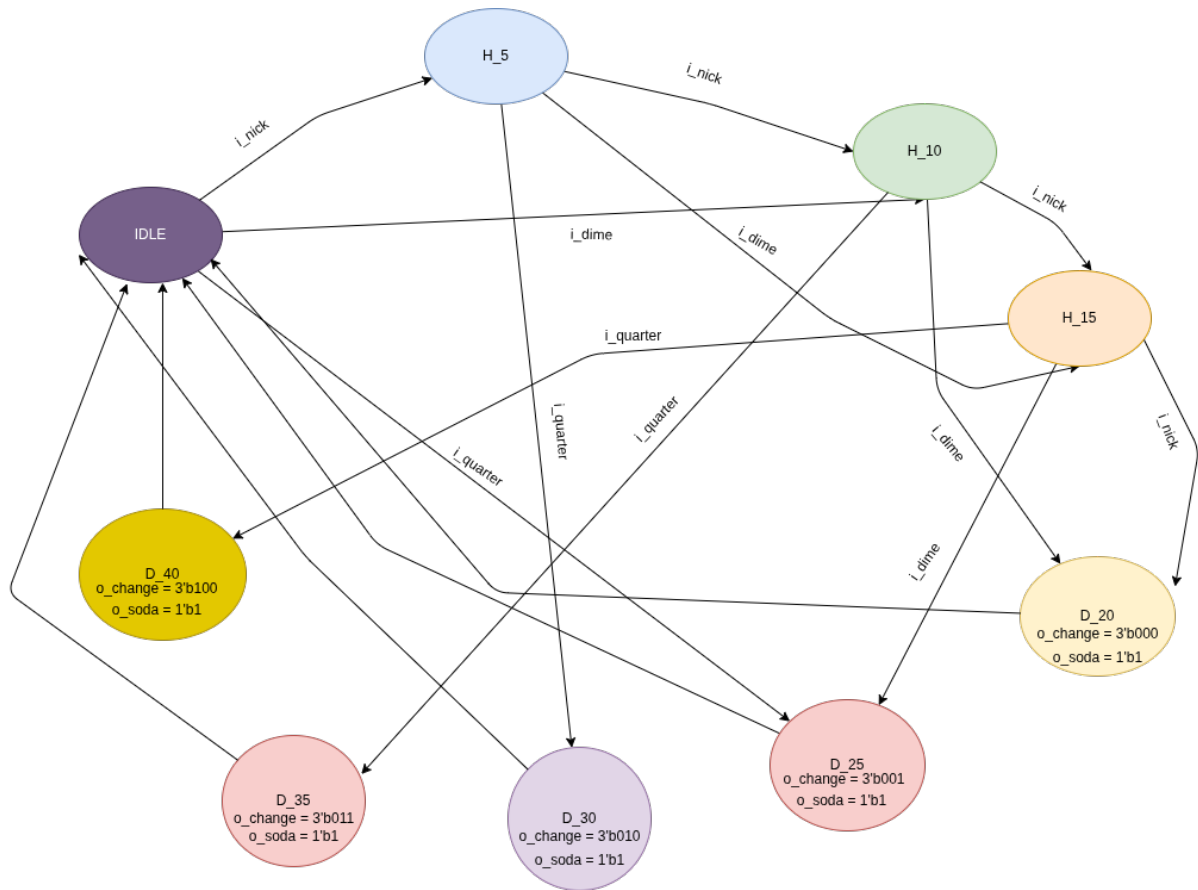


Figure 2: FSM Control

The system accepts sequential insertions of 5¢, 10¢, and 25¢ coins. Once the accumulated value reaches or exceeds the 20¢ threshold, the FSM asserts the dispense signal and outputs the corresponding change. Notably, the state machine was optimized from an initial 13-state design down to a minimized 9-state architecture. This state reduction not only conserves hardware resources but also enhances design clarity, and it has been fully verified to comply with the functional specifications.

3 Verification Architecture

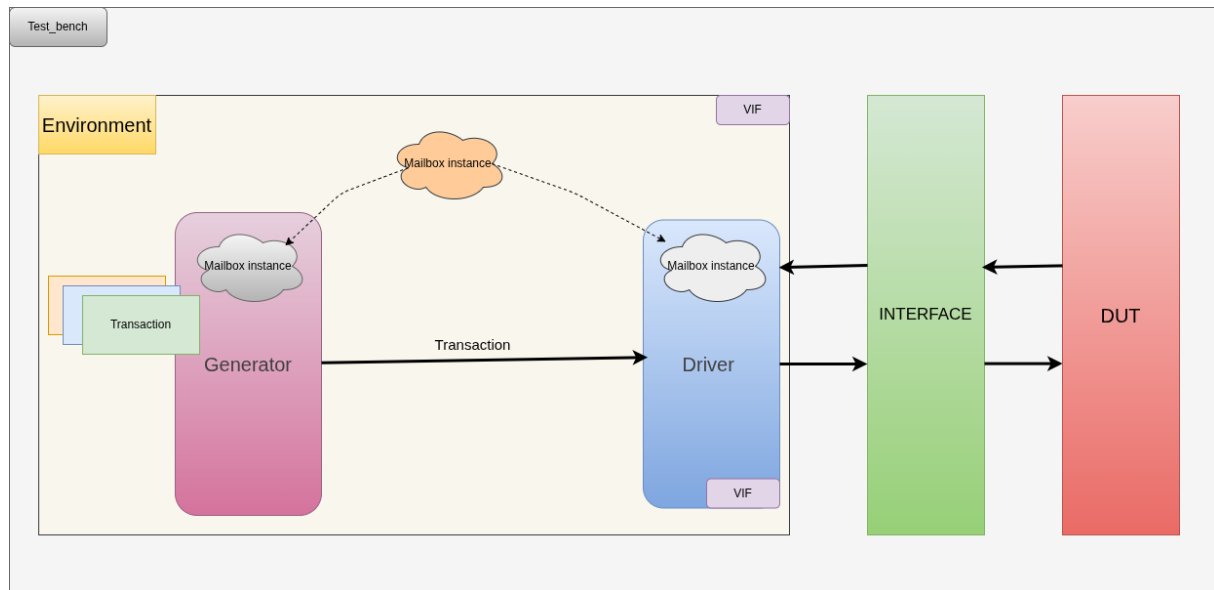


Figure 3: FSM Control

To verify this design, a reusable verification environment was developed using SystemVerilog Object-Oriented Programming (OOP) principles. The environment comprises fundamental components: Transaction, Generator, Driver, Environment, Interface, and the Device Under Test (DUT). To generate scenarios and stimulate the DUT, a handshake mechanism was implemented between the Generator and the Driver using a Mailbox, which operates as a FIFO buffer. Specifically, the Generator instantiates and packs transaction objects, then pushes them into the Mailbox. Concurrently, the Driver retrieves these transactions, translates them into pin-level logic signals, and drives the DUT via the Interface. This automated process continues until all transactions are exhausted. Finally, the functional correctness of the design is evaluated by analyzing simulation waveforms under two methodologies: Directed Testing and Constrained Random Testing.

The above diagram clearly illustrates the hierarchical structure and the handshake mechanism within the verification environment:

- **Transaction:** A data class that encapsulates the coin-insertion signals.
- **Generator:** The block responsible for generating stimulus (supporting both directed and random scenarios) and pushing it into the Mailbox.
- **Driver:** The physical execution block that retrieves data from the Mailbox via a common communication port and drives the DUT's signal pins through a Virtual Interface (VIF).
- **DUT & Interface:** The hardware block under test, connected to the environment via signal wires defined within the Interface.

4 Test Scenarios & Simulation Results

This section evaluates the functional correctness of the Vending Machine FSM through two main testing methodologies. The simulation ensures that the design adheres to the specifications and handles various credit states reliably.

4.1 Directed Test

The directed test phase focuses on exhaustive state coverage, verifying every terminal state from D_20 to D_40. Each test case validates a specific coin insertion sequence and checks if the system provides the correct dispense signal and exact change.

Case	Coin Sequence (Stimulus)	Total	Target	Change	Log
1	10¢ + 10¢	20¢	D_20	0¢ (3'b000)	PASSED
2	25¢	25¢	D_25	5¢ (3'b001)	PASSED
3	5¢ + 25¢	30¢	D_30	10¢ (3'b010)	PASSED
4	10¢ + 25¢	35¢	D_35	15¢ (3'b011)	PASSED
5	5¢ + 10¢ + 25¢	40¢	D_40	20¢ (3'b100)	PASSED

Table 4: Summary of Directed Test Scenarios

As detailed in Table 4.1, five distinct scenarios were crafted to systematically evaluate every terminal state (from D_20 to D_40). These cases act as functional boundary checks, ensuring that regardless of the coin combination used to reach or exceed the 20¢ threshold, the FSM accurately asserts the dispense signal and calculates the correct change. The consistent **PASSED** results across all scenarios confirm the logical correctness of the combinational output block.

Log File

```
[TEST CASE 1] Target: 20c (State D_20)
The transaction have been driven to 0
[1100] [test_direct] N:0 D:1 Q:0
Put the transaction is done
[1100] [DRIVER] Driver bat dau run Mailbox...
[1100] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:1 Q:0
The transaction have been driven to 0
[1450] [test_direct] N:0 D:1 Q:0
Put the transaction is done
[1450] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:1 Q:0
[1650] PASSED: Soda=1, Change=000

[TEST CASE 2] Target: 25c (State D_25)
The transaction have been driven to 0
[2150] [test_direct] N:0 D:0 Q:1
Put the transaction is done
[2150] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:0 Q:1
[2350] PASSED: Soda=1, Change=001

[TEST CASE 3] Target: 30c (State D_30)
The transaction have been driven to 0
[2850] [test_direct] N:1 D:0 Q:0
Put the transaction is done
[2850] [DRIVER] Nhan duoc Trans ngau nhien: N:1 D:0 Q:0
The transaction have been driven to 0
[3250] [test_direct] N:0 D:0 Q:1
Put the transaction is done
[3250] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:0 Q:1
[3450] PASSED: Soda=1, Change=010

... (Remaining test cases omitted for brevity)
```

Table 5: Directed Test Simulation Log Output (Partial)

Waveform Analysis

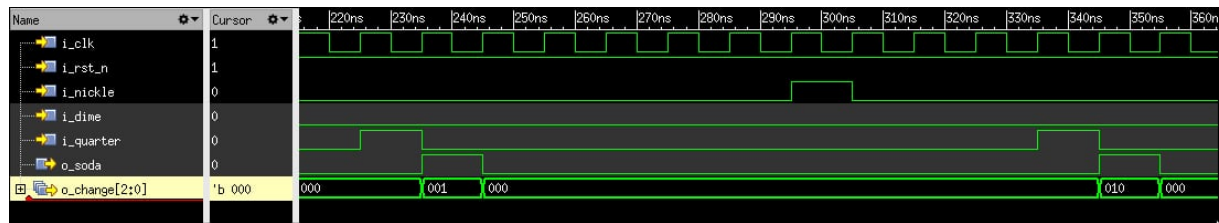


Figure 4: Waveform Of Direct Test

As illustrated in Figure 1, the simulation waveform vividly captures a segment of the Directed Test phase, specifically demonstrating Test Case 2 and Test Case 3. In the first transaction, the assertion of the `i_quarter` signal (25c) immediately triggers the FSM to pull the `o_soda` signal high and output 3'b001 (5c) on the change bus at the next clock edge. Subsequently, the second transaction shows a sequential insertion of a nickel (`i_nickle`) and a quarter (`i_quarter`), accumulating to 30c. The system correctly responds by asserting `o_soda` and providing exactly 10c in change (3'b010). This timing diagram perfectly validates the synchronous nature of the Moore FSM, proving that the outputs respond accurately without any combinational glitches.

4.2 Constrained Random Test

The random testing phase was executed to evaluate the FSM's robustness under unpredictable input sequences. The table below summarizes the configuration and the final verification results for this phase.

Parameter	Configuration / Simulation Result
Test Methodology	Constrained Random Verification (CRV)
Transaction Count	20 Automated Transactions
Randomization Rule	One-hot coin selection (<code>i_nickle</code> , <code>i_dime</code> , <code>i_quarter</code>)
Data Communication	Mailbox-based Handshake (Synchronous FIFO)
Testing Objective	Stress testing and exhaustive state transition coverage
Simulation Time	500ns (Total duration)
Final Status	PASSED (100% functional accuracy)

Table 6: Summary of Parameters and Results for the Random Test Phase

To evaluate the robustness and stability of the FSM, a stress test with 20 random transactions was conducted. This phase utilizes the `randomize()` function with one-hot constraints to simulate unpredictable user behavior. The mailbox-based handshake mechanism successfully synchronized the data flow between the Generator and Driver without any data loss or logic errors.

Log File

```
[RANDOM PHASE] Running 20 random transactions...
The time that We want to count is: 20
[6850] [test_random] N:1 D:0 Q:0
[6850] [DRIVER] Nhan duoc Trans ngau nhien: N:1 D:0 Q:0
[6860] [test_random] N:0 D:0 Q:1
[6870] [test_random] N:0 D:0 Q:1
[6880] [test_random] N:0 D:1 Q:0
[6890] [test_random] N:0 D:1 Q:0
... (Generator continues pushing transactions into Mailbox)

[7250] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:0 Q:1
[7650] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:0 Q:1
[8050] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:1 Q:0
[8450] [DRIVER] Nhan duoc Trans ngau nhien: N:0 D:1 Q:0
... (Driver continuously popping transactions from Mailbox)

[14050] [DRIVER] Nhan duoc Trans ngau nhien: N:1 D:0 Q:0
[14450] [DRIVER] Nhan duoc Trans ngau nhien: N:1 D:0 Q:0

-- SIMULATION FINISHED --
Simulation complete via $finish(1) at time 1495 NS + 1
```

Table 7: Constrained Random Test Simulation Log Output (Partial)

Waveform Analysis

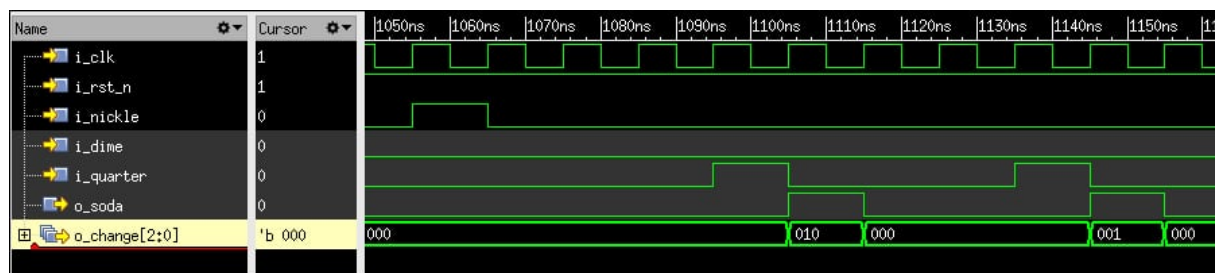


Figure 5: Waveform Of Random Test

As depicted in Figure 2, the waveform captures a dynamic segment of the Constrained Random Test phase, highlighting the FSM’s reliability under unpredictable input streams. In the observed time frame, the testbench randomly generates an `i_nickle` (5¢) followed by an `i_quarter` (25¢). The system accurately accumulates this arbitrary combination to 30¢, synchronously asserting the `o_soda` signal and dispensing exactly 10¢ in change (3'b010). Immediately following this cycle, another random transaction injects a single quarter. The FSM seamlessly processes this independent 25¢ input, pulling `o_soda` high again and outputting 3'b001 (5¢) on the change bus. This timing diagram confirms the design’s robustness, proving that the Mailbox-driven random stimuli are processed flawlessly with strict adherence to the Moore FSM synchronous timing constraints.

5 Conclusion

Through this project, we have significantly improved our RTL coding skills and deepened our collective understanding of Finite State Machine (FSM) architectures, enabling us to approach similar hardware design problems with greater confidence. Most notably, this lab provided our team with valuable hands-on experience in developing a reusable Object-Oriented Programming (OOP) verification environment using SystemVerilog, which proved to be a highly effective methodology for fundamental design verification. Ultimately, the comprehensive simulation results from both directed and random test phases confirm that the Vending Machine design strictly adheres to the functional specifications and the project has been successfully completed.